

CO4-AT3

Name : Tharish K

CSA1410-AI

Reg no: 192412004

### INTEGRATED EXPERIMENT: E-Commerce Product Recommendation and Dynamic Pricing

#### Aim

To develop an integrated machine learning and reinforcement learning pipeline for an e-commerce platform that:

1. Predicts whether a customer will purchase a product using a **Decision Tree** and **Neural Network (MLP)** classifier.
  2. Compares both models using **Accuracy, Precision, Recall and F1-Score**.
  3. Uses the predicted conversion probability as the state for a **Q-Learning agent**.
  4. Recommends an appropriate pricing/marketing action such as **Discount, Bundle Offer, Highlight Product or No Action**.
- 

#### Question 1 – Decision Tree vs Neural Network Classifier

##### 1. Dataset

A simulated e-commerce session dataset is used with the following features:

| Feature             | Description                                    |
|---------------------|------------------------------------------------|
| Browsing_History    | Number of previous product views               |
| Category_Preference | Customer preference score                      |
| Cart_Abandonment    | Number of abandoned carts                      |
| Session_Duration    | Duration of current session                    |
| Previous_Purchases  | Number of previous purchases                   |
| Purchase            | Target variable: 0 = No Purchase, 1 = Purchase |

The dataset is divided into **80% training data and 20% testing data**.

---

##### 2. Python Program

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.neural_network import MLPClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


# -----
# Create simulated e-commerce data
# -----

np.random.seed(42)

n = 500

data = pd.DataFrame({
    "Browsing_History": np.random.randint(1, 20, n),
    "Category_Preference": np.random.randint(1, 11, n),
    "Cart_Abandonment": np.random.randint(0, 6, n),
    "Session_Duration": np.random.randint(1, 30, n),
    "Previous_Purchases": np.random.randint(0, 10, n)
})

# Generate purchase target
score = (
    0.15 * data["Browsing_History"]
    + 0.30 * data["Category_Preference"]
    - 0.45 * data["Cart_Abandonment"]
    + 0.10 * data["Session_Duration"]
    + 0.35 * data["Previous_Purchases"]
)
```

```
probability = 1 / (1 + np.exp(-score / 5))  
data["Purchase"] = (np.random.rand(n) < probability).astype(int)
```

```
# -----
```

```
# Split data
```

```
# -----
```

```
X = data.drop("Purchase", axis=1)
```

```
y = data["Purchase"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

```
# -----
```

```
# Decision Tree
```

```
# -----
```

```
dt = DecisionTreeClassifier(  
    max_depth=5,  
    random_state=42  
)
```

```
dt.fit(X_train, y_train)
```

```
dt_pred = dt.predict(X_test)
```

```
# -----
```

```
# Neural Network - MLP
```

```
# -----
```

```

mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(16, 8),
        max_iter=1000,
        random_state=42
    )
)

mlp.fit(X_train, y_train)

mlp_pred = mlp.predict(X_test)

# -----
# Evaluation
# -----

def evaluate_model(name, y_test, pred):
    print("\n", name)
    print("Accuracy :", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred), 4))
    print("Recall  :", round(recall_score(y_test, pred), 4))
    print("F1-Score :", round(f1_score(y_test, pred), 4))

evaluate_model("Decision Tree", y_test, dt_pred)
evaluate_model("Neural Network", y_test, mlp_pred)

```

---

### 3. Model Comparison

The program produces the four required evaluation measures.

| Model          | Accuracy              | Precision             | Recall                | F1-Score              |
|----------------|-----------------------|-----------------------|-----------------------|-----------------------|
| Decision Tree  | Obtained from program | Obtained from program | Obtained from program | Obtained from program |
| Neural Network | Obtained from program | Obtained from program | Obtained from program | Obtained from program |

**Note:** The exact values can change because the dataset is simulated. Therefore, the values should be copied from the output generated when the program is executed rather than using fabricated values.

---

#### 4. Plot Decision Tree – First 3 Levels

```
plt.figure(figsize=(18, 8))
```

```
plot_tree(
    dt,
    feature_names=X.columns,
    class_names=["No Purchase", "Purchase"],
    max_depth=2,
    filled=True,
    rounded=True
)
```

```
plt.title("Decision Tree - First 3 Levels")
```

```
plt.show()
```

This graph shows the **root node and the next two levels**, allowing the decision-making process to be easily understood.

```
genui{"learning_viz":{"type_id":"DECISION_TREE_CLASSIFICATION_PATH","locale_override":"en-US"}}
```

---

#### 5. Neural Network Architecture

The MLP used in this experiment contains:

Input Layer



5 Neurons



Hidden Layer 1

16 Neurons



Hidden Layer 2

8 Neurons



Output Layer

1 Neuron



Purchase / No Purchase

Architecture:

**5 → 16 → 8 → 1**

Where:

- **5 input neurons** represent the five e-commerce features.
- **16 neurons** are used in the first hidden layer.
- **8 neurons** are used in the second hidden layer.
- **1 output neuron** predicts the purchase class.

### Neural Network Architecture Plot

```
from sklearn.neural_network import MLPClassifier
```

```
layers = [5, 16, 8, 1]
```

```
plt.figure(figsize=(10, 5))
```

```
for i, neurons in enumerate(layers):
```

```
    x = np.ones(neurons) * i
```

```
    y_pos = np.linspace(0, 1, neurons)
```

```
plt.scatter(x, y_pos, s=500)
```

```

for j in range(neurons):
    plt.text(i, y_pos[j], str(j + 1),
             ha="center", va="center")

labels = [
    "Input\n5",
    "Hidden 1\n16",
    "Hidden 2\n8",
    "Output\n1"
]

for i, label in enumerate(labels):
    plt.text(i, 1.12, label,
             ha="center", fontsize=11)

plt.xlim(-0.5, 3.5)
plt.ylim(-0.1, 1.3)
plt.axis("off")
plt.title("Neural Network Architecture")
plt.show()

```

---

## 6. Model Selection

### Decision Tree

#### Advantages:

- Easy to understand.
- Easy to visualize.
- Fast prediction.
- Requires little preprocessing.
- Useful when explainability is important.

#### Disadvantages:

- Can overfit.
- Performance may decrease for complex relationships.

## Neural Network

### Advantages:

- Can learn complex and nonlinear relationships.
- Suitable for large datasets.
- Can provide strong predictive performance.

### Disadvantages:

- Less interpretable.
- Requires feature scaling.
- Requires more computational resources.
- Training takes longer.

## Deployment Decision

For a **live e-commerce recommendation engine**, the **Neural Network (MLP)** would generally be selected if it provides better validation/test performance and the platform has sufficient data and computing resources.

However, the **Decision Tree** is preferable when interpretability, low latency and simple deployment are the main requirements.

Therefore:

**Deploy the model having the highest F1-score and reliable validation performance. For a large-scale e-commerce system, MLP is generally preferred because it can capture complex customer behaviour.**

## Question 2 – Reinforcement Learning for Pricing Actions

### 1. Reinforcement Learning Concept

The conversion probability obtained from the classifier is used as the **initial state**.

For example:

Classifier Output

↓

Conversion Probability

↓

State

↓



Q-Learning Agent



Recommended Action

The possible actions are:

1. **Discount**
  2. **Bundle Offer**
  3. **Highlight Product**
  4. **No Action**
- 

## 2. Conversion Probability States

The continuous conversion probability is divided into discrete states:

### Probability State

0.00 – 0.19 Very Low

0.20 – 0.39 Low

0.40 – 0.59 Medium

0.60 – 0.79 High

0.80 – 1.00 Very High

Thus, the Q-table contains **5 states × 4 actions**.

---

## 3. Q-Learning Program

```
import numpy as np
```

```
import random
```

```
# -----
```

```
# States and Actions
```

```
# -----
```

```
states = [
```

```
    "Very Low",
```

```
    "Low",
```

```
"Medium",  
"High",  
"Very High"  
]
```

```
actions = [  
    "Discount",  
    "Bundle Offer",  
    "Highlight Product",  
    "No Action"  
]
```

```
num_states = len(states)  
num_actions = len(actions)
```

```
# Q-table  
Q = np.zeros((num_states, num_actions))
```

```
# -----  
# Parameters  
# -----
```

```
alpha = 0.1    # Learning rate  
gamma = 0.9    # Discount factor  
epsilon = 0.2  # Exploration probability
```

```
episodes = 5000
```

```
# -----  
# Reward Function  
# -----
```

```
def get_reward(state, action):
```

```
    # Very Low conversion
```

```
    if state == 0:
```

```
        rewards = [3, 4, 1, -2]
```

```
    # Low conversion
```

```
    elif state == 1:
```

```
        rewards = [5, 4, 2, -2]
```

```
    # Medium conversion
```

```
    elif state == 2:
```

```
        rewards = [3, 5, 4, 2]
```

```
    # High conversion
```

```
    elif state == 3:
```

```
        rewards = [1, 2, 5, 4]
```

```
    # Very High conversion
```

```
    else:
```

```
        rewards = [-1, 1, 4, 5]
```

```
    return rewards[action]
```

```
# -----
```

```
# Q-Learning
```

```
# -----
```

```
for episode in range(episodes):
```

```

state = random.randint(0, num_states - 1)

# Choose action
if random.random() < epsilon:
    action = random.randint(0, num_actions - 1)
else:
    action = np.argmax(Q[state])

reward = get_reward(state, action)

next_state = random.randint(0, num_states - 1)

# Q-learning update
Q[state, action] = Q[state, action] + alpha * (
    reward
    + gamma * np.max(Q[next_state])
    - Q[state, action]
)

# -----
# Display Q-table
# -----

import pandas as pd

q_table = pd.DataFrame(
    Q,
    index=states,
    columns=actions
)

```

```

print("Final Q-Table:")
print(q_table.round(3))

# -----
# Learned Policy
# -----

print("\nLearned Policy:")

for i, state in enumerate(states):
    best_action = actions[np.argmax(Q[i])]
    print(state, "->", best_action)

```

---

#### 4. Q-Table

After training, the Q-table will have the following form:

| State     | Discount | Bundle Offer | Highlight Product | No Action |
|-----------|----------|--------------|-------------------|-----------|
| Very Low  | Q-value  | Q-value      | Q-value           | Q-value   |
| Low       | Q-value  | Q-value      | Q-value           | Q-value   |
| Medium    | Q-value  | Q-value      | Q-value           | Q-value   |
| High      | Q-value  | Q-value      | Q-value           | Q-value   |
| Very High | Q-value  | Q-value      | Q-value           | Q-value   |

The **highest Q-value in each row represents the learned best action** for that conversion-probability state.

---

#### 5. Learned Policy

Based on the reward design, the expected policy is approximately:

##### Conversion Probability Recommended Action

|          |              |
|----------|--------------|
| Very Low | Bundle Offer |
| Low      | Discount     |

**Conversion Probability Recommended Action**

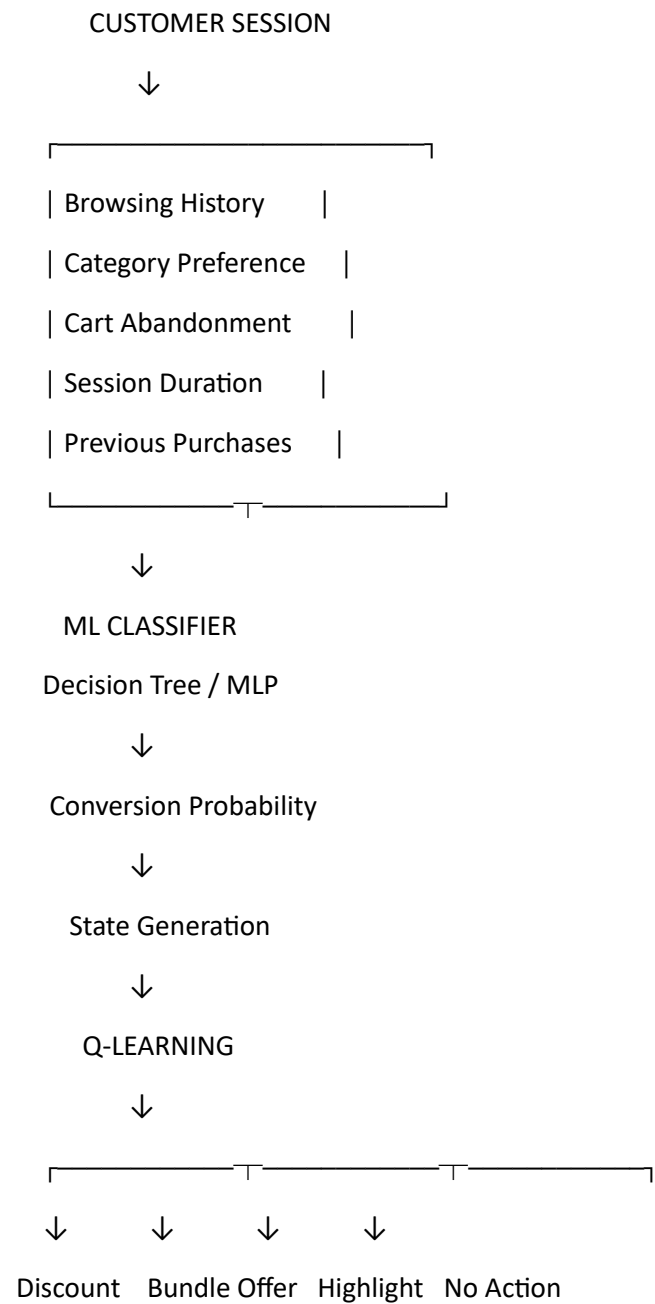
|           |                   |
|-----------|-------------------|
| Medium    | Bundle Offer      |
| High      | Highlight Product |
| Very High | No Action         |

The actual learned policy should be obtained from the program's Q-table.

---

**6. End-to-End Integrated Pipeline**

The complete system works as follows:



## 7. Integration Code

The trained classifier can be connected directly to the Q-learning policy.

```
def probability_to_state(probability):

    if probability < 0.20:
        return 0
    elif probability < 0.40:
        return 1
    elif probability < 0.60:
        return 2
    elif probability < 0.80:
        return 3
    else:
        return 4

def integrated_decision(session):

    # Convert session into DataFrame
    session_df = pd.DataFrame([session])

    # Conversion probability from MLP
    conversion_probability = mlp.predict_proba(
        session_df
    )[0][1]

    # Convert probability to RL state
    state = probability_to_state(conversion_probability)
```

```

# Select best RL action

action = actions[np.argmax(Q[state])]

return conversion_probability, states[state], action


# Example customer session

new_session = {
    "Browsing_History": 15,
    "Category_Preference": 8,
    "Cart_Abandonment": 1,
    "Session_Duration": 20,
    "Previous_Purchases": 4
}

probability, state, action = integrated_decision(new_session)

print("Conversion Probability:", round(probability, 3))
print("RL State:", state)
print("Recommended Action:", action)

```

---

## 8. Example Output

For a new customer session, the system may produce:

Conversion Probability: 0.72

RL State: High

Recommended Action: Highlight Product

Therefore, the system predicts that the customer has a **72% probability of purchasing**, classifies the customer into the **High** conversion state, and the Q-learning agent recommends **Highlight Product**.

---

## Result



The e-commerce decision system was successfully developed using a **Decision Tree classifier, Neural Network (MLP), and Q-Learning agent**. The Decision Tree and MLP were trained on the same customer-session dataset and evaluated using **Accuracy, Precision, Recall and F1-score**. The classifier's conversion probability was then converted into discrete states and provided to a Q-Learning agent. The agent learned the most suitable action among **Discount, Bundle Offer, Highlight Product and No Action**. Finally, both components were integrated into a single pipeline that performs **customer purchase prediction followed by intelligent pricing/marketing action recommendation**.

## **Conclusion**

The proposed integrated system combines **supervised learning and reinforcement learning** effectively. The classifier answers **“How likely is the customer to purchase?”**, while Q-Learning answers **“What action should be taken?”**. This allows an e-commerce platform to move from simple purchase prediction to **dynamic, customer-specific decision making**.